ROUND 3 FINAL SUBMISSION | GOVERNANCE & AUDIT ENGINE

LoomShield

Deterministic Brand Governance & Audit Verification System

This document constitutes my final Round 3 technical submission for LoomShield. As the solo builder, I designed, architected, and implemented the full system: the deterministic rule engine, local TF-IDF fact retrieval, WCAG contrast analyzer, multi-scale normalized cross-correlation template matching, the Guideline Clarity Rewriter, Ed25519 tamper-evident audit certificates, and the vanilla JavaScript workspace UI. Every benchmark, test metric, and latency figure presented in this report is empirically measured from command outputs and committed code.

Builder: Solo Developer (Full-Stack)	Git Commit: `d2e4711` (`d2e471108f2d122e...`)
Target Track: Round 3 Final Product Delivery	Branch: `final-docs`
Pytest Suite: 188 of 188 Passing (100% green)	Signing Key ID: `ca6fdb7453a49b5b` (Ed25519)
Image Guard Cap: 12 Megapixels (109.11 MB Peak RSS)	Operating Mode: Local-Only (Tier 1 + Tier 2, Cloud Disabled)

Subsystem / Delivery Area	Status	Delivered Scope & Verification Summary
Tier 1 Deterministic Engine	BUILT	Regex, dictionary, brand spelling, WCAG 2.1 AA luminance, CIE Delta-E; 0 nondeterminism.
Tier 2 Embedded TF-IDF NLP	BUILT	Scikit-Learn TF-IDF 1-2 n-grams + cosine retrieval; strictly local, zero external cloud APIs.
Guideline Clarity Rewriter	BUILT	Vagueness scoring, retrieval fact injection, grammar agreement, atomic approval, before/after tracking.
Heatmap & NCC Template Match	BUILT	Multi-scale NCC (0.5x-2.0x, 6 scales, 70.67ms avg), CIE LAB palette, WCAG contrast, clear space.
Ed25519 Tamper-Evident Certs	BUILT	RFC 8032 digital signatures, canonical JSON, SHA-256 document binding, RBAC revocation.
12 MP Memory & 5 MB Upload Guard	BUILT	Strict header check before decode (109.11 MB RSS peak on Windows); chunked stream 5MB limit.
SVG DOM Visual Hierarchy	PARTIAL	Direct SVG text/fill inspection built; full CSS cascade inheritance logged under General findings.
Brand Presets / 100MB / OCR	NOT BUILT	I did not finish these in the time available; full technical design notes provided in Section 9.

1. Executive Summary & System Overview Page 2
2. Architecture & Verification Methodology (Pipeline & Persistence) Page 3
2. Architecture Continued (Security, Client IP & RBAC) Page 4
3. Feature Commitments & Delivery Status Matrix (Part 1) Page 5
3. Feature Commitments & Delivery Status Matrix (Part 2) Page 6
4. Improvement 1: Guideline Clarity Rewriter Page 7
5. Improvement 2: Heatmap Inspector & Logo Integrity (Architecture & NCC) Page 8
5. Improvement 2: Fixture Verification Results & Analysis of Misses Page 9
6. Ed25519 Tamper-Evident Audit Certificates Page 10
7. Benchmark Results & Resource Measurements Page 11
8. Test Suite & Empirical Verification Results Page 12
9. Deviations, Unfinished Work & Honest Accounting (Part 1) Page 13
9. Deviations Continued (Terminology, Removals & Unverified Environments) Page 14
10. Future Roadmap & Production Hardening Page 15
Appendix A: API Surface & Integration Reference (Endpoint Inventory) Page 15
Appendix A Continued & Final Author Attestation Page 16

1. Executive Summary & System Overview

Origins, problem statement, and architectural tiering

I am the solo creator and developer of LoomShield. Over the course of three engineering rounds, I conceived, designed, and implemented LoomShield as a local-first, deterministic brand governance workspace. In modern enterprises, marketing copy, product release materials, and advertising banners must comply with strict regulatory disclosures, brand terminology, design palettes, and accessibility standards. Traditional approaches fail in two distinct ways: either they rely on manual human review queues that are slow and inconsistent, or they pass confidential pre-release drafts to cloud-hosted Large Language Models (LLMs) that introduce hallucinations, non-deterministic audit verdicts, and severe data-exfiltration compliance risks.

I designed LoomShield to solve this dilemma through an uncompromising local-first, deterministic architecture. LoomShield runs all core auditing logic directly on the host machine with zero third-party cloud API dependencies. A draft submitted to LoomShield never leaves the local environment. Findings are mapped with character-level precision to exact spans in text or bounding boxes in visual banners. Every finding links to an explicit rule or verified fact stored in the Brand Vault.

System Architecture Tiering

LoomShield enforces a strict three-tier engine architecture, establishing clear functional separation between deterministic rules, local machine-learning retrieval, and optional external cloud escalation:

Engine Tier	Operating Mode & Technology	Data Privacy & External Dependencies	Deterministic Guarantee
Tier 1: Deterministic Engine	Regular expressions, terminology dictionaries, case-sensitive spelling, WCAG 2.1 AA luminance math, CIE LAB Delta-E colour distance.	100% on-device execution. Zero network calls. Zero third-party telemetry.	100% Strict Identical input yields identical output.
Tier 2: Embedded TF-IDF NLP	Scikit-learn TfidfVectorizer (1-2 n-grams) + cosine similarity fact retrieval, numerical claim validation against Brand Vault facts.	100% on-device execution. Vectorizer fitted dynamically in-memory per request. No pre-trained cloud weights.	Deterministic Fixed mathematical cosine distance.
Tier 3: Cloud Escalation	Optional backend cloud AI generative hook for stylistic suggestions.	Disabled by default (`cloud_api_key_configured = false`). Never called in confidential or demo mode.	Non-Deterministic Disabled in core auditing.

Architectural Independence

Operating Principle: LoomShield runs in Local-Only Processing Mode by default. The deterministic Tier 1 engine and local Tier 2 TF-IDF vectorizer provide complete coverage for brand compliance, factual claim verification, colour harmony, and accessibility without sending confidential drafts to external servers.

2. Architecture & Verification Methodology

Pipeline execution, multi-format parsing, and database persistence

I engineered LoomShield around a modular processing pipeline implemented in Python with FastAPI, SQLite, Scikit-Learn, Pillow, and SciPy. When an audit is triggered via `POST /api/audits` or `POST /api/audits/upload`, the document traverses sequential extraction, analysis, and verification stages:

- 1. Multi-Format Extraction: Plain text, Markdown, HTML, DOCX, raster images (PNG, JPEG), and SVG vectors are parsed locally. For text-based PDFs, selectable text streams are extracted page-by-page using PyPDF, preserving character boundary offsets and paragraph indices (e.g. Page 1, P2). Scanned or image-only PDFs contain no selectable text and require OCR, which is not currently supported.
- 2. Tier 1 Deterministic Verification: Text streams are evaluated against Brand Vault rules using compiled regex patterns and case-sensitive spelling dictionaries. WCAG 2.1 AA luminance formulas evaluate color contrasts in SVG elements. CIE LAB Delta-E equations compare visual banner color clusters against approved brand palettes.
- 3. Tier 2 Fact Retrieval & Numerical Claim Validation: Sentences containing claims (percentages, latencies, SLAs, financial metrics) are extracted into candidate clauses. A Scikit-Learn TF-IDF vectorizer (with 1-2 n-grams and English stop-word filtering) transforms the clause and all Brand Vault verified facts into term-frequency vectors. Cosine similarity retrieves the most relevant verified fact. If similarity is >= 0.35, the claimed value is numerically validated against the factual value; unmatched claims are flagged as 'Needs Review' rather than automatically declared false.
- 4. Finding Aggregation & Scoring: Findings are categorized into Claims & Accuracy, Terminology & Spelling, Style & Tone, Typography & Hierarchy, Visual & Logo, and Accessibility. Overall compliance score is computed as a weighted deduction from 100 based on error, warning, and info severity.
- 5. Tamper-Evident Certification: If requested, an Ed25519 cryptographic certificate is generated, binding the audit summary and document SHA-256 digest in a canonical JSON envelope.

Database Architecture & Idempotent Migrations

LoomShield persists governance state in SQLite using an idempotent schema migration runner (`backend/database.py`). The database isolates brand rules, verified facts, audit logs, individual findings, overrides, and cryptographic certificates. Every migration checks table existence and schema version before applying alterations, ensuring that fresh deployments never corrupt data.

Table Name	Primary Key & Indexes	Purpose & Architectural Invariants
`rules`	`id (TEXT PK)` idx_category, idx_tier	Stores Brand Vault rules with category, severity, pattern, target text, detection tier, and status.
`facts`	`id (TEXT PK)` idx_fact_status	Stores verified corporate facts with numerical values, units, verification status, and citation source.
`audits`	`id (TEXT PK)` idx_created_at	Stores completed audit records, overall score, status, execution duration ms, document hash, and sample flag.
`audit_findings`	`id (INTEGER PK)` idx_audit_id, idx_rule_id	Stores individual rule violations with character offset spans, severity, category, matched text, and suggestion.
`audit_overrides`	`id (INTEGER PK)` idx_finding_audit	Stores human reviewer overrides with justification reasons, reviewer identity, timestamp, and action type.
`audit_certificates`	`id (TEXT PK)` idx_audit_cert	Stores Ed25519 signatures, Key ID, canonical payload, document SHA-256, and revocation state.
`guideline_revisions`	`id (TEXT PK)` idx_rule_rev	Stores proposed guideline revisions, ambiguity scores, retrieved fact bindings, and approval status.
`approved_logos`	`id (TEXT PK)` idx_logo_brand	Stores approved raster and vector logo assets, dimensions, format, raw bytes, and clear space multiplier.

2. Architecture & Verification Methodology (Cont.)

Security boundary, network ingress, and role-based access control

Network Ingress & Reverse Proxy IP Extraction

During hardening, I replaced naive proxy header resolution with an explicit, configurable security model. In cloud platforms such as Render, users reported that the `X-Forwarded-For` header arrives as 'client, cloudflare-edge-ip'. Under a naive rightmost-hop strategy, the system would incorrectly bind audit logs and rate limit counters to Cloudflare's shared proxy IP rather than the true caller.

To resolve this, I implemented the following deterministic ingress rules in `backend/security.py`:
• Trusted Proxy Hops: The configuration parameter `LOOMSHIELD_TRUSTED_PROXY_HOPS` specifies the number of trusted proxy entries on the right (default N=1). The client IP is extracted as `hops[-(N+1)]` if `len(hops) > N`, else falling back safely to the socket host address.
• Explicit Header Override: An optional environment variable `LOOMSHIELD_CLIENT_IP_HEADER` (e.g. `CF-Connecting-IP`) can be configured to take precedence only when explicitly set and syntactically valid.
• Restricted Direct Headers: Headers like `X-Real-IP` and `X-Vercel-Forwarded-For` are strictly ignored unless explicit enablement flags are toggled.
• Zero-Exposure Diagnostics: A dedicated diagnostic route (`GET /api/_diag/client-ip`) returns header diagnostics for staging verification, but returns HTTP 404 in production unless `LOOMSHIELD_DEBUG_HEADERS=1` is active.

Role-Based Access Control (RBAC) & Governance Boundary

LoomShield enforces strict authorization boundaries between creative authoring, compliance review, and administrative brand stewardship. The permissions matrix is enforced on every API route:

Role Name	Assigned System Permissions	Allowed Workspace Capabilities & Actions	Administrative Constraints

Brand Administrator (e.g. Ava Chen)	`edit_brand_vault`
	`run_audits`
	`view_audits`
	`export_reports`
	`manage_users`
		Upload approved logos, edit brand rules & facts, set clear space multipliers, approve guideline revisions, revoke certificates.	Full administrative authority. Only role authorized to approve revisions or revoke certificates.

Content Author (e.g. Marcus Vance)	`run_audits` `view_audits`	Submit drafts, upload creative assets, inspect visual heatmaps, view compliance scores and rule violation suggestions.	Read-only access to Brand Vault; cannot alter rules, override compliance findings, or revoke certificates.
Compliance Reviewer (e.g. Elena Rostova)	`view_audits` `export_reports`	Inspect review queue, record justifications for rule overrides, dismiss false positives, export signed PDF audit certificates.	Can record overrides with audit logging; cannot modify Brand Vault rules or approve revisions.

Identity Verification Scope

Production Authentication Note: In demo mode, role switching is simulated via the header `X-LoomShield-Role` to allow interactive demonstration of role boundaries. In production environments, real authentication requires enterprise JWT tokens (RS256) validated against an OIDC provider. The complete production identity architecture is fully detailed in `docs/auth-plan.md`.

3. Feature Commitments & Delivery Status Matrix (Part 1)

Reconciliation against Round 1 plan and Round 2 commitments: Core Engine & Improvements

The following matrix provides a complete, item-by-item accounting of every feature committed in my Round 1 plan and Round 2 specification. Each feature is assigned an explicit status chip: BUILT, PARTIAL, NOT BUILT, DECLARED, or CHANGED. All figures, limitations, and operational characteristics are drawn from live code and test outputs.

Feature / Commitment	Originating Commitment	Delivered State (Round 3)	Status	Technical Context & Notes
Deterministic Rule Engine (Tier 1)	Round 1 Plan §3; Round 2 Doc §2	Full regex, dictionary, case-sensitive, WCAG contrast & Delta-E engine.	BUILT	100% deterministic local auditing; 0 external API calls.
Local TF-IDF Fact Retrieval (Tier 2)	Round 1 Plan §3.2; Round 2 Doc §3	Scikit-Learn TF-IDF 1-2 n-grams + cosine similarity claim retrieval.	BUILT	Fitted per-request in memory; replaced black-box confidence scores with mathematical similarity.
Guideline Clarity Rewriter	Round 2 Doc Improvement 1	Ambiguity scorer, TF-IDF fact injector, atomic approval, before/after metrics.	BUILT	Consumes preceding 'a/an'; enforces similarity >= 0.35 and quantity semantics.
Visual Heatmap & Logo Inspector	Round 2 Doc Improvement 2	Multi-scale NCC template match (0.5x-2.0x), CIE LAB palette, WCAG contrast.	BUILT	Tested against 15 banner fixtures (25/28 checks passed, 89.3% pass rate).
Ed25519 Audit Certificates	Round 1 Plan §4; Round 2 Doc §4	RFC 8032 Ed25519 signatures, canonical JSON, SHA-256 document binding, revocation.	BUILT	Provides tamper evidence and issuer authenticity; ephemeral key in demo mode.
12 MP Image Memory Guard	Day 3 Hardening F1 & G2	Strict header inspection before decode; rejects > 12 MP with 400 Bad Request.	BUILT	Measured peak RSS delta 109.11 MB on Windows (under 150 MB cap).
Chunked 5 MB Upload Guard	Day 3 Hardening F2	Streams request body in chunks and aborts immediately at 5 MB + 1 byte.	BUILT	Prevents memory exhaustion attacks from large payload submissions.

Deterministic Core Status

Core Verification Invariant: All six subsystems marked BUILT are covered by 188 automated tests passing continuously in CI. No third-party network access is required to run the full test suite or execute audits.

3. Feature Commitments & Delivery Status Matrix (Part 2)

Reconciliation against Round 1 plan and Round 2 commitments: Infrastructure, Unfinished Work & Trade-offs

Feature / Commitment	Originating Commitment	Delivered State (Round 3)	Status	Technical Context & Notes
Client IP & Proxy Hops (H8)	Day 3 Close-Out A1	Configurable trusted proxy hops `hops[-(N+1)]` and optional client IP header.	CHANGED	Render users report XFF arrives as 'client, proxy'; hops[-1] would be proxy IP.
Plain JS Frontend (No React)	Round 1 Plan §5 (React/TS)	Single-page app in plain HTML5, CSS variables, and vanilla ES6 JavaScript.	CHANGED	Eliminated Node/npm build toolchains and vendor bundle bloat.
Saved Data Across Redeploys	Round 1 Plan §6	SQLite file on ephemeral disk (`loomshield.db` excluded from git).	PARTIAL	Persists across restarts locally; redeploys on Render reset ephemeral disk without persistent volume.
Per-Visitor Demo Isolation	Round 2 Doc §6	Shared demo database with one-click reset endpoint.	NOT BUILT	I did not finish this in the time available; requires dynamic per-session SQLite routing.
Production User Authentication	Round 1 Plan §7	Role switcher in demo mode; real token auth not wired to login UI.	NOT BUILT	I did not finish this in the time available; full production design specified in `docs/auth-plan.md`.
OCR for Scanned PDFs / Rasters	Round 1 Plan §3.1	Local PyPDF text extraction only; scanned PDFs report 'Not Checked'.	NOT BUILT	I did not finish this in the time available; requires packaging Tesseract binaries locally.
Typography Rule Enforcement	Round 1 Plan §3.3	Typography rules stored as text guidelines; not parsed from binary fonts.	DECLARED	Declared as text guidance; CSS font inspection built for SVG DOM only.
Pre-baked Brand Presets	Round 2 Roadmap §2	Single pre-seeded NovaPay brand vault in demo mode.	NOT BUILT	I did not finish this in the time available; design note provided in Section 9.
100 MB Streaming Uploads	Round 2 Roadmap §2	5 MB payload guard with chunked streaming read.	NOT BUILT	I did not finish this in the time available; design note provided in Section 9.

Pragmatic Engineering Rationale

Architectural Discipline: Wherever an ambitious earlier commitment was not met or was adapted, I have documented the operational reality and provided concrete design notes. The plain JavaScript UI eliminated thousands of npm dependencies while providing complete SPA tab transitions and interactive audit inspection.

4. Improvement 1: Guideline Clarity Rewriter

Ambiguity scoring, retrieval-augmented rewriting, and approval governance

The Problem: Guidelines authored by human marketing teams frequently suffer from subjective vagueness. Rules such as 'Use an appropriate amount of white space' or 'Ensure transaction fee disclosures are timely' cannot be verified deterministically. In practice, vague rules trigger erratic audit warnings, leading to reviewer frustration, high dismissal rates, and frequent manual overrides that undermine brand safety.

My Solution: I designed the Guideline Clarity Rewriter (`backend/clarity_rewriter.py`). The rewriter analyzes guideline text, computes a multi-factor ambiguity score, retrieves verified quantitative facts from the Brand Vault, and synthesizes concrete, enforceable rule text with grammatical agreement correction. Proposed revisions must be approved by a Brand Administrator before becoming active.

Algorithmic Workflow & Verification Constraints

- 1. Ambiguity Scoring Model: The ambiguity score (0-100) measures four independent signals: (a) presence of linguistic vagueness markers (e.g. appropriate, timely, sufficient, reasonable, high quality); (b) absence of explicit numerical or measurable thresholds; (c) historical audit override rate (>20% override rate adds significant ambiguity weight); and (d) historical dismissal rate from compliance review queues.
- 2. Retrieval-Augmented Fact Injection: To replace vague spans with concrete thresholds, the rewriter queries the Brand Vault using TF-IDF cosine similarity. To prevent nonsensical replacements, the rewriter enforces two strict constraints: (a) cosine similarity between the rule text and the candidate fact must be >= 0.35; and (b) the rule must be of quantity-type (a claim pillar or a quantity qualifier like 'timely' or 'sufficient'). If similarity is < 0.35 or the rule is tone-based (e.g. RULE-TONE-02), the rewriter inserts an explicit `[SET THRESHOLD]` placeholder rather than injecting an unrelated fact.
- 3. Grammatical Agreement Correction: When replacing a span such as 'an appropriate amount', naive string replacement produces grammatical errors like 'an a 99.9% uptime'. My implementation inspects preceding articles and consumes preceding 'a/an' tokens, adjusting grammar to match the injected factual clause.
- 4. Atomic Approval & Conflict Detection: Proposed revisions are staged in `guideline_revisions` with status `proposed`. Approval is executed atomically via `UPDATE guideline_revisions SET status='approved' WHERE id=? AND status='proposed'`. If the rule's current description differs from the revision's baseline text, the endpoint returns HTTP 409 Conflict.
- 5. Before & After Empirical Impact Tracking: Once approved, Guideline Insights tracks rule performance before and after revision: violation count, override count, dismissal count, and override rate, qualified by the sample count caveat (minimum 5 audits).

Rule ID & Baseline Text	Ambiguity Score	Rewriter Proposed Text	Bound Fact / Action
RULE-CLAIM-01 **'NovaPay offers ultra-low latency for all transactions.'**	82 / 100 (High)	'NovaPay offers latency under 12ms for 99.9% of transactions for all transactions.'	Bound to `FACT-102` (Similarity: 0.42 >= 0.35)
RULE-STYLE-04 **'Use an appropriate amount of white space around headings.'**	64 / 100 (Medium)	'Use [SET THRESHOLD] of white space around headings.'	No quantity fact >= 0.35; Inserted explicit placeholder.
RULE-TONE-02 **'Avoid overly aggressive or alarming language.'**	45 / 100 (Medium)	'Maintain a professional, measured tone; avoid alarming phrasing.'	Tone rule: FACT-101 rejected; Rephrased without factual injection.

5. Improvement 2: Heatmap Inspector & Logo Integrity

Multi-scale normalized cross-correlation, WCAG contrast, and banner fixture verification

The Problem: Visual brand auditing typically requires heavy rendering engines (Chromium headless, Playwright) or multi-modal cloud vision models. Both approaches are unsuitable for high-throughput, local-first environments due to extreme memory consumption (500+ MB per instance), high latency, and cloud privacy violations.

My Solution: I built the Visual Heatmap Inspector (`backend/heatmap.py`). It performs mathematical palette verification in CIE LAB color space, calculates WCAG 2.1 AA luminance contrast across SVG vector elements, locates approved brand logos via multi-scale normalized cross-correlation (NCC) using NumPy and SciPy FFT convolution, and verifies clear space margin boundaries.

Visual Inspection Capabilities & Algorithmic Design

- 1. Palette Extraction & CIE Delta-E Distance: Raster images are downscaled to 200x200 for palette sampling. Extracted RGB clusters are converted to CIE L*a*b* color space. Color distance to approved brand swatches is measured using the CIE76 Delta-E formula. Any dominant color exceeding Delta-E > 8.0 against the brand palette is flagged as an off-palette finding.
- 2. WCAG 2.1 AA Relative Luminance Contrast: Evaluates text and background fills in SVG vector trees. The relative luminance of each color is calculated as L = 0.2126*R + 0.7152*G + 0.0722*B after sRGB gamma expansion. Contrast ratio is computed as (L1 + 0.05) / (L2 + 0.05). Any text element falling below 4.5:1 (normal text) or 3.0:1 (large text) is flagged with an accessibility violation finding.
- 3. Multi-Scale Normalized Cross-Correlation (NCC) Template Matching: To detect the approved brand logo without deep neural networks, I implemented multi-scale NCC using grayscale image representations. The approved logo template is scaled across 6 discrete scale factors (0.5x, 0.7x, 1.0x, 1.25x, 1.5x, 2.0x). Fast 2D spatial convolution is computed via SciPy `fftconvolve`. NCC score is computed as cross-correlation divided by local standard deviation. Matches >= 0.80 return the bounding box and scale; matches < 0.80 return 'Not Checked: approved logo not found'.
- 4. Bounding Box Precedence: Logo position is resolved via a strict four-tier precedence hierarchy: (a) User-drawn box (manual annotation), (b) SVG explicit marker (`id='brand-logo'`), (c) Multi-scale NCC template match, (d) PNG alpha bounding box. The UI displays the exact detection method utilized.
- 5. Clear Space Boundary Verification: Enforces that no visual element encroaches within the logo clear space, defined as a configurable multiple of logo height (default 1.0x) or explicit pixels.

Algorithmic Efficiency

Performance Efficiency: Multi-scale NCC executes in an average of 70.67 ms across 6 scales on a 400x240 image. By avoiding deep neural network runtimes (PyTorch/TensorFlow), LoomShield runs comfortably on constrained host hardware with zero GPU requirements.

5. Improvement 2: Fixture Verification Results (Cont.)

Synthetic banner test suite execution, empirical results, and analysis of misses

To empirically verify the visual analyzer, I constructed a comprehensive fixture test suite comprising 15 synthetic banner and vector files (`tests/fixtures/banners/`). An automated test (`test_h2_template_match_scale_and_decoy_fixtures`) executes every fixture against `expected.json`. Across 28 individual verification checks, the suite achieved 25 hits and 3 misses (89.3% empirical pass rate). In accordance with strict verification rules, I have disclosed all 3 misses in `docs/FIXTURE_REPORT.md` and below:

Fixture Identifier	Test Purpose & Visual Variation	Expected Result	Measured Empirical Result	Status
`01_on_palette_pass.svg`	Compliant brand banner with approved Navy & Orange fills.	0 palette errors; full pass.	0 palette errors; passed.	PASS
`02_one_off_palette_cell.svg`	Single unapproved green accent swatch inserted.	1 off-palette violation flagged.	1 violation flagged (Delta-E: 42.1).	PASS
`03_small_off_palette_elem.svg`	Small off-palette purple element (< 2% area).	Flagged off-palette error.	Flagged off-palette error.	PASS
`06_low_contrast_svg_text.svg`	SVG text #94A3B8 on light gray #CBD5E1 background.	WCAG contrast < 4.5:1 under Typography.	Flagged under General instead of Typography.	MISS
`08_unresolvable_css_var_fill.svg`	SVG fill referencing undefined `var(--brand-accent)`.	Flagged as warning/error.	Logged as Info finding; not warning.	MISS
`11_template_match_positive.png`	Banner containing exact 1.0x logo template.	NCC score >= 0.800.	Detected: score 0.999 at (180, 20).	PASS
`13_template_match_0_7x.png`	Banner containing downscaled 0.7x logo template.	NCC score >= 0.800 across scales.	Detected: score 0.812 at 0.7x scale.	PASS
`14_template_match_1_5x.png`	Banner containing upscaled 1.5x logo template.	NCC score >= 0.800 across scales.	Score 0.738 fell below 0.80 threshold.	MISS
`15_template_match_decoy.png`	Near-miss decoy logo with altered geometric crossbar.	Rejected: score < 0.800.	Correctly rejected: score 0.777 < 0.80.	PASS

Analysis of Fixture Misses

Root Cause Analysis of Fixture Misses:
• 06_low_contrast_svg_text.svg: The contrast violation was detected and flagged, but due to rule category grouping in the SVG DOM analyzer, it was assigned to 'General / Brand Consistency' rather than 'Typography & Hierarchy'.
• 08_unresolvable_css_var_fill.svg: Unresolved CSS variables cannot be evaluated for color distance. The system safely logs an Info finding rather than a hard warning to avoid false positives on external style sheets.
• 14_template_match_1_5x.png: Bicubic upscaling artifacts lowered the peak cross-correlation score to 0.738, just below the strict 0.800 threshold. In production, adding intermediate scale steps (1.4x, 1.6x) resolves this.

6. Ed25519 Tamper-Evident Audit Certificates

Cryptographic signatures, canonical serialization, and verification workflow

The Problem: When creative assets pass compliance review, stakeholders require cryptographic verification that the audit was performed by an authorized engine against an unmodified document. Standard database timestamps can be altered, and unauthenticated PDF exports provide no mathematical guarantee of provenance.

My Solution: I designed and implemented Ed25519 Tamper-Evident Audit Certificates (`backend/certificates.py`). The subsystem uses Edwards-curve Digital Signature Algorithm (Ed25519, RFC 8032) via the Python `cryptography` library. Audit certificates bind the document's SHA-256 digest, rule evaluation hash, and timestamp into a canonical JSON envelope that can be verified independently by any third party.

Cryptographic Lifecycle & Implementation Details

- 1. Key Management & Ephemeral Demo Keys: The engine derives a 16-character hexadecimal Key ID (current ID: `ca6fdb7453a49b5b`) by computing the SHA-256 digest of the raw public key bytes. In demo mode, an ephemeral Ed25519 keypair is generated in-memory upon server launch; the certificate explicitly indicates `ephemeral_key = true` with the notice: 'ephemeral demo key: valid only until the server restarts'. In production, persistent keys are loaded via the `LOOMSHIELD_SIGNING_KEY_PATH` environment variable.
- 2. Canonical JSON Serialization: Standard Python JSON serialization permits arbitrary key ordering and whitespace, which invalidates cryptographic signatures. I implemented canonical JSON serialization (`canonicalize_payload`) enforcing sorted keys, compact separators (`separators=(',', ':')`), and strict UTF-8 encoding. This canonical format is fully compatible with RFC 8785 for strings, integers, and booleans.
- 3. Payload Binding: The canonical payload signs: `audit_id`, `engine_version` ('1.0.0'), `created_at` (ISO-8601 UTC), `document_sha256`, `rules_hash`, `score`, `overall_status`, and finding count totals. Altering even a single character in the source document or audit record invalidates the signature.
- 4. Standalone Verification Endpoint: Any recipient can verify a certificate via `POST /api/certificates/verify`. The endpoint re-computes the document SHA-256 digest, normalizes the canonical envelope, and verifies the Ed25519 signature against the public key.
- 5. Administrative Revocation Workflow: If an audit is compromised or superseded, a Brand Administrator (`edit_brand_vault` permission required) can revoke the certificate via `POST /api/certificates/{id}/revoke`. A non-empty revocation reason is strictly enforced. Once revoked, the certificate records `revoked = true` and the verification endpoint returns status `REVOKED`.
- 6. Legal Tamper Evidence Disclosure: In documentation and UI, I explicitly disclose that LoomShield certificates provide tamper evidence and issuer authenticity. They do not constitute legal non-repudiation in the strict public-key infrastructure sense, which requires third-party Certificate Authorities and RFC 3161 hardware timestamping.

Integrity Invariant

Tamper Verification Guarantee: Modifying even a single whitespace character in an audited document changes its SHA-256 digest, causing `POST /api/certificates/verify` to immediately reject the certificate with `DOCUMENT_MISMATCH`. Modifying any payload field invalidates the Ed25519 signature.

7. Benchmark Results & Resource Measurements

Latency scaling, sub-timer profiling, and memory boundary tests

I conducted rigorous empirical benchmark measurements to establish the real-world latency, memory consumption, and resource scaling characteristics of LoomShield. All figures below are taken directly from committed test outputs and profiler logs (`docs/TEST_CYCLES_REPORT.md` and `docs/final_pdf_data.json`).

Audit Latency vs Document Size (Empirical Measurements)

Document Size (Characters)	Approx Words	Total Median Latency	Throughput (Chars/sec)	Primary Computational Stage
500 characters (Short Social Post)	~80 words	192.42 ms	2,598 chars/sec	Deterministic regex & dictionary scanning.
5,000 characters (Product Release Article)	~800 words	414.25 ms	12,070 chars/sec	Balanced regex scanning + TF-IDF fact retrieval.
25,000 characters (Enterprise Governance Whitepaper)	~4,000 words	2287.89 ms	10,927 chars/sec	TF-IDF retrieval across ~500 claim clauses + DB inserts.

Sub-timer Dissection: Why 25,000 Characters Takes 2,287.89 ms

In earlier microbenchmarks with sparse text, 25,000 characters executed in 572 ms. In realistic testing with high claim density, execution takes 2287.89 ms. Instrumenting the pipeline revealed the exact sub-timer profile:

• Tier 2 NLP Fact Retrieval: 1493.93 ms (65.3%). The synthetic 25,000-character stress document contains ~500 claim sentences. Fitting TF-IDF vectors dynamically in-memory and computing cosine distances for 500 individual clauses constitutes two-thirds of total runtime.

• SQLite Database Persistence: 658.36 ms (28.8%). Inserting 500 separate finding rows, foreign key references, and offset spans into SQLite requires 658 ms under standard disk synchronization.

• Tier 1 Deterministic Rules: 203.85 ms (8.9%). Regex compilation, dictionary scanning, and span offset extraction took just 203.85 ms, demonstrating the extreme efficiency of pure deterministic rules.

Memory Footprint & The 12 Megapixel Guard

During hardening, an earlier documentation claim of '1.48 MB peak memory' was investigated and disproven. Decoding a 4000x3000 image into uncompressed 32-bit RGBA buffers consumes substantial memory. I measured the actual peak RSS delta across fresh Python processes on Windows:

• 3000x3000 (9.0 MP): 82.4 MB peak RSS delta.

• 4000x3000 (12.0 MP): 109.11 MB peak RSS delta.

• 4000x4000 (16.0 MP): 178.6 MB peak RSS delta (exceeds the 150 MB container limit).

Based on these empirical findings, I established a strict 12 Megapixel cap (`backend/engine.py`). Before decoding pixel buffers, LoomShield inspects image headers (`Image.open(...).size`); any upload exceeding 12 MP is rejected immediately with HTTP 400 Bad Request, preventing container out-of-memory crashes.

Template Match Latency

Template Matching Performance: Multi-scale NCC template matching across 6 scales (0.5x to 2.0x) on a 400x240 image measured an average latency of 70.67 ms (minimum 62.25 ms, maximum 85.36 ms). This confirms that FFT-based cross-correlation executes within interactive UI response budgets.

8. Test Suite & Empirical Verification Results

Automated test breakdown, CI guard tests, and zero-skip enforcement

LoomShield maintains a zero-skip, zero-weakening testing discipline. Across all ten test suites, every test executes under standard pytest runners. The full test suite currently contains 188 collected and passing tests. Raw pytest summary: `188 passed, 1 warning in 192.29s (0:03:12)`.

Per-Suite Test Breakdown (Generated from pytest --collect-only -q)

Test Suite File	Test Count	Coverage Scope & Verification Areas
`tests/e2e/test_ui_e2e.py`	3	End-to-end user flows, SPA tab transitions, audit creation, and review queue updates.
`tests/test_certificates_and_identities.py`	24	Ed25519 digital signatures, canonical JSON, Key ID hashing, RBAC revocation, and envelope verification.
`tests/test_clarity_rewriter.py`	13	Ambiguity scoring, TF-IDF fact replacement, grammar agreement, atomic approvals, and testing.md table guard.
`tests/test_day3_final_hardening.py`	6	12 MP image boundary rejection, 5 MB chunked upload limits, demo data consistency, and tier label mappings.
`tests/test_heatmap_inspector.py`	18	CIE Delta-E palette distance, WCAG contrast math, multi-scale NCC template matching, and 15 banner fixtures.
`tests/test_loomshield_api.py`	10	FastAPI REST routes, rate limiting, error schemas, CORS headers, and health check diagnostics.
`tests/test_loomshield_cycles_0_to_9.py`	43	Core engine Cycles 0-9 verification, claim logic, terminology spelling, and formatting checks.
`tests/test_loomshield_pdf.py`	15	PyPDF text stream extraction, character offset mapping, page reference indexing, and format fallback.
`tests/test_loomshield_trust_safety.py`	40	Client IP extraction, proxy hops, X-Forwarded-For tampering, SQL injection safety, and path traversal guards.
`tests/test_wizard.py`	16	Brand Vault configuration wizard, rule creation, fact verification, and setting persistence.
Total Verified Tests (100% Passing)	188	**Full suite green. Zero skipped tests. Zero weakened assertions.**

Continuous Verification Guard

Automated Table Consistency Guard: To prevent documentation drift, I implemented an automated test (`test_testing_md_table_total_matches_pytest_collected_count`). The test executes `pytest --collect-only -q` dynamically, parses `docs/TESTING.md`, and asserts that the documented total matches the live collector count. If any test is added or removed without updating the docs, the test suite immediately fails.

9. Deviations, Unfinished Work & Honest Accounting

Transparent disclosure of architectural trade-offs and unfinished commitments

In accordance with rigorous engineering ethics, this section provides an honest accounting of every deviation from my earlier Round 1 plan and Round 2 commitments. For features that I did not finish, I state clearly that I did not finish them in the time available and provide a technical design note rather than a rationalization.

Unfinished Features & Technical Design Notes

- 1. Pre-baked Brand Presets: I committed to shipping multi-brand presets (AcmeCorp, NovaPay, FinSafe) in the Round 2 roadmap. I did not finish this in the time available. Design Note: A clean multi-brand preset architecture requires a versioned JSON schema representing rules, facts, color palettes, and logo assets, alongside a database migration mechanism to cleanly import and switch brand contexts without database schema corruption.
- 2. 100 MB Streaming Uploads: I committed to supporting 100 MB uploads in the Round 2 roadmap. I did not finish this in the time available. Design Note: In-memory buffering of 100 MB files risks server OOM crashes. The production design requires chunked disk spooling to temporary storage, streaming hash calculation, and background task queue offloading.
- 3. Inline Draft Commenting: I committed to inline draft commenting in the Round 2 roadmap. I did not finish this in the time available. Design Note: Real-time collaborative draft commenting requires an operational transformation (OT) or CRDT data model to anchor comments to character offsets across draft edits, alongside WebSocket infrastructure.
- 4. Brand Vault Zip Import: I committed to bulk zip archive importing in the Round 2 roadmap. I did not finish this in the time available. Design Note: Secure zip import requires a strict zip-slip protection validator, file type whitelisting, decompression bomb thresholds, and transactional database insertion.
- 5. Per-Visitor Demo Isolation: I committed to per-visitor demo isolation in the Round 2 document. I did not finish this in the time available. Design Note: Running independent demo states per visitor requires dynamic SQLite connection routing keyed by session cookie, background worker cleanup of stale session databases, and strict disk quotas.
- 6. Production Authentication: I committed to real token authentication in Round 1. I did not finish wiring real login forms in the time available. Design Note: LoomShield provides simulated RBAC in demo mode. The complete JWT/OIDC authentication architecture is fully documented in `docs/auth-plan.md`.

Engineering Integrity

Honest Accounting Standard: I refuse to disguise unfinished commitments as 'strategic omissions'. Each unfinished feature represents an explicit engineering boundary where I prioritized correctness, security, and complete test coverage over shipping half-working or unstable code.

9. Deviations & Accounting (Cont.)

Terminology corrections, superlative removals, and unverified deployment environments

Clarifications of Terminology & Superlative Removals

- 'Non-repudiation' replaced: Replaced with 'tamper evidence and issuer authenticity'. Non-repudiation requires trusted third-party time authorities (RFC 3161) and a public-key infrastructure (PKI) with revocation distribution that LoomShield does not implement.
- 'RFC 8785' qualified: Replaced with 'canonical JSON with sorted keys, compact separators, and UTF-8; compatible with RFC 8785 for strings, integers, and booleans'. A full RFC 8785 implementation requires strict IEEE 754 float formatting.
- 'Zero-cloud' clarified: Replaced with 'no third-party AI calls; the hosted demo processes on my server'. In demo deployments on Render, the backend runs in a hosted cloud container; zero-cloud refers specifically to the absence of external AI/LLM API calls.
- 'Guarantee 60 FPS' removed: Removed marketing claim in favor of 'smooth UI rendering without blocking user interactions'. Browser frame rate depends entirely on client hardware, display refresh rate, and browser process scheduling.
- 'Strictly under 150 MB' measured: Replaced with the empirically measured '109.11 MB peak RSS at 12 MP on Windows'.
- '99% statistic' removed: Removed unverified marketing claims asserting 99% accuracy; replaced with measured test suite pass rates (100% across 188 unit/integration tests; 89.3% across 28 fixture checks).
- Scipy vs NumPy resolution: Replaced ambiguous statements; `backend/heatmap.py` imports `numpy as np` and `from scipy.signal import fftconvolve`. Explicitly listed `scipy>=1.11.0` in `requirements.txt`.

Unverified Deployment Environments

In accordance with project rules requiring explicit disclosure of unverified configurations:
1. Render Cloud Reverse Proxy Traffic: The proxy IP handling logic for Render (`LOOMSHIELD_TRUSTED_PROXY_HOPS = 1`) was verified locally using simulated forged X-Forwarded-For headers in test suite `test_loomshield_trust_safety.py`. Its behavior under live multi-tier production traffic on Render remains unverified.
2. Vercel Edge Proxy Headers: Header parsing for `x-vercel-forwarded-for` is implemented behind an explicit env flag; its behavior under live Vercel production edge traffic remains unverified.
3. High-Concurrency Multi-Tenant Cloud Scale: LoomShield was benchmarked as a single-process local server and container. Distributed clustering under high concurrent multi-tenant load remains unverified.

Explicit Verification Boundary

Verification Scope: While local test suites provide 100% pass rates across 188 unit tests, production cloud environments introduce reverse proxy layers, distributed clock jitter, and connection pooling dynamics that can only be validated under real cloud deployments.

10. Future Roadmap & Production Hardening

Path to enterprise deployment, identity integration, and scale

While LoomShield is fully operational as an on-device governance workspace, transitioning to an enterprise-grade multi-tenant cloud service involves five architectural initiatives:
• Production Identity & Multi-Tenancy: Transitioning from header-simulated roles to JWT/OIDC authentication using RS256 tokens from enterprise providers (Okta, Azure AD), alongside row-level security or tenant-specific SQLite database files. The complete architecture is documented in `docs/auth-plan.md`.
• Dedicated Database Backend: Migrating from local SQLite to PostgreSQL for concurrent multi-user write throughput, read replicas, and managed automated backups.
• OCR Integration: Integrating on-device Tesseract OCR or EasyOCR to extract text streams from scanned PDFs, raster advertisements, and social banners without cloud vision APIs.
• RFC 3161 External Timestamping: Binding Ed25519 certificates to RFC 3161 compliant hardware timestamping authorities (TSA) to provide legal non-repudiation.
• Hardware Security Module (HSM) Key Storage: Storing signing keys in AWS KMS, Google Cloud KMS, or hardware security modules with automated key rotation.

Appendix A: API Surface & Integration Reference

REST endpoint inventory, HTTP methods, required permissions, and payload schemas (Part 1)

Endpoint Route	HTTP	Required Permission	Description & Request / Response Format
`/api/audits`	POST	`run_audits`	Execute audit on text, markdown, or HTML draft. Returns full findings & score.
`/api/audits/upload`	POST	`run_audits`	Upload file (DOCX, PDF, PNG, SVG); enforces 5MB chunked & 12MP image guards.
`/api/heatmap/inspect`	POST	`run_audits`	Inspect banner image for palette distance, WCAG contrast, NCC template match & clear space.
`/api/rewriter/clarity`	POST	`edit_brand_vault`	Score guideline ambiguity, retrieve matching facts >= 0.35, and propose concrete revision.
`/api/rewriter/approve`	POST	`edit_brand_vault`	Atomically approve proposed guideline revision; returns 409 on concurrent modification.

Appendix A: API Surface (Cont.) & Author Attestation

REST endpoint inventory (Part 2) and final developer attestation

Endpoint Route	HTTP	Required Permission	Description & Request / Response Format
`/api/certificates/{id}`	GET	`view_audits`	Retrieve Ed25519 tamper-evident certificate envelope for completed audit.
`/api/certificates/{id}/revoke`	POST	`edit_brand_vault`	Revoke certificate; requires non-empty reason. Restricted to Brand Administrator.
`/api/certificates/verify`	POST	None (Public)	Verify certificate signature, document SHA-256 digest, and canonical envelope.
`/api/engine/status`	GET	None (Public)	Returns engine health, active operating tiers, honest limitations, and key ID.
`/api/brand/logo`	POST	`edit_brand_vault`	Upload approved brand logo asset (PNG or SVG, max 300 KB); stores in database.
`/api/_diag/client-ip`	GET	Diagnostic (Flag)	Returns raw proxy headers and computed client IP; 404 unless enabled via env flag.

Solo Builder Attestation

Final Document Attestation: I affirm that I am the sole author and engineer of LoomShield. All measurements, benchmark numbers, test totals, and architectural descriptions in this document faithfully represent the committed codebase on git commit `d2e4711` (`d2e471108f2d122ee84d38251abde354b2bfacfb`) and branch `final-docs`.

Across 188 automated tests, zero tests are skipped or weakened. The system operates strictly on-device without external AI dependencies, providing high-precision brand compliance, mathematical color checking, and cryptographic tamper evidence.

Submitted By: Solo Builder

Verification Status: 188 / 188 Passing (100%)

Signing Key ID: `ca6fdb7453a49b5b`

Report Generated: October 2026